

Database Systems (CS2005)

Week – 15

Instructor: **Basit Ali**

CHAPTER 21

Concurrency Control Techniques

Introduction

- Concurrency control protocols
 - Set of rules to guarantee serializability
- Two-phase locking protocols
 - Lock data items to prevent concurrent access
- Timestamp
 - Unique identifier for each transaction

21.1 Two-Phase Locking Techniques for Concurrency Control

- Lock
 - Variable associated with a data item describing status for operations that can be applied
 - One lock for each item in the database
- Binary locks
 - Two states (values)
 - Locked (1)
 - Item cannot be accessed
 - Unlocked (0)
 - Item can be accessed when requested

Two-Phase Locking Techniques for Concurrency Control (cont'd.)

- Transaction requests access by issuing a `lock_item(X)` operation

```
lock_item(X):  
  B:  if LOCK(X) = 0          (*item is unlocked*)  
        then LOCK(X) ← 1      (*lock the item*)  
      else  
        begin  
          wait (until LOCK(X) = 0  
                and the lock manager wakes up the transaction);  
          go to B  
        end;  
unlock_item(X):  
  LOCK(X) ← 0;                (* unlock the item *)  
  if any transactions are waiting  
    then wakeup one of the waiting transactions;
```


Two-Phase Locking Techniques for Concurrency Control (cont'd.)

- Lock table specifies items that have locks
- Lock manager subsystem
 - Keeps track of and controls access to locks
 - Rules enforced by lock manager module
- At most one transaction can hold the lock on an item at a given time
- Binary locking too restrictive for database items

Two-Phase Locking Techniques for Concurrency Control (cont'd.)

- If the simple binary locking scheme described is used, then every transaction must obey the following rules:
 1. A transaction T must issue the operation `lock_item(X)` before any `read_item(X)` or `write_item(X)` operations are performed in T .
 2. A transaction T must issue the operation `unlock_item(X)` after all `read_item(X)` and `write_item(X)` operations are completed in T .
 3. A transaction T will not issue a `lock_item(X)` operation if it already holds the lock on item X .¹
 4. A transaction T will not issue an `unlock_item(X)` operation unless it already holds the lock on item X .

Two-Phase Locking Techniques for Concurrency Control (cont'd.)

- **Shared/exclusive or read/write locks or multi-mode locks**
 - Read operations on the same item are not conflicting
 - Must have exclusive lock to write
 - Three locking operations
 - `read_lock(X)`
 - `write_lock(X)`
 - `unlock(X)`

Figure 21.2 Locking and unlocking operations for two-mode (read/write, or shared/exclusive) locks

```
read_lock(X):
B: if LOCK(X) = "unlocked"
    then begin LOCK(X) ← "read-locked";
        no_of_reads(X) ← 1
    end
    else if LOCK(X) = "read-locked"
        then no_of_reads(X) ← no_of_reads(X) + 1
    else begin
        wait (until LOCK(X) = "unlocked"
            and the lock manager wakes up the transaction);
        go to B
    end;

write_lock(X):
B: if LOCK(X) = "unlocked"
    then LOCK(X) ← "write-locked"
    else begin
        wait (until LOCK(X) = "unlocked"
            and the lock manager wakes up the transaction);
        go to B
    end;

unlock (X):
if LOCK(X) = "write-locked"
    then begin LOCK(X) ← "unlocked";
        wakeup one of the waiting transactions, if any
    end
else if LOCK(X) = "read-locked"
    then begin
        no_of_reads(X) ← no_of_reads(X) - 1;
        if no_of_reads(X) = 0
            then begin LOCK(X) = "unlocked";
                wakeup one of the waiting transactions, if any
            end
    end
end;
```

Scenario

T1 only wants to read X → needs Shared Lock (S-lock)
T2 wants to write X → needs Exclusive Lock (X-lock)

Time	T1 (Reader)	T2 (Writer)
t1	lock-S(X) granted	-
t2	read(X)	lock-X(X) → blocked
t3	unlock(X)	still waiting
t4	-	lock-X(X) granted
t5	-	write(X)
t6	-	unlock(X)

Two-Phase Locking Techniques for Concurrency Control (cont'd.)

- Lock conversion
 - Transaction that already holds a lock allowed to convert the lock from one state to another
- Upgrading
 - Issue a read_lock operation then a write_lock operation
- Downgrading
 - Issue a read_lock operation after a write_lock operation

- Using binary locks or read/write locks in transactions, as described earlier, does not guarantee serializability of schedules on its own.

	T_1	T_2
(a)	<pre> read_lock(Y); read_item(Y); unlock(Y); write_lock(X); read_item(X); X := X + Y; write_item(X); unlock(X); </pre>	<pre> read_lock(X); read_item(X); unlock(X); write_lock(Y); read_item(Y); Y := X + Y; write_item(Y); unlock(Y); </pre>

(b) Initial values: $X=20$, $Y=30$

Result serial schedule T_1
followed by T_2 : $X=50, Y=80$

Result of serial schedule T_2
followed by T_1 : $X=70, Y=50$

(c)

	T_1	T_2
Time ↓	read_lock(Y); read_item(Y); unlock(Y);	read_lock(X); read_item(X); unlock(X); write_lock(Y); read_item(Y); $Y := X + Y$; write_item(Y); unlock(Y);
	write_lock(X); read_item(X); $X := X + Y$; write_item(X); unlock(X);	

Result of schedule S:
X=50, Y=50
(nonserializable)

Figure 21.3
Transactions that do not obey two-phase locking.
(a) Two transactions T_1 and T_2 . (b) Results of possible serial schedules of T_1 and T_2 . (c) A nonserializable schedule S that uses locks.

Guaranteeing Serializability by Two-Phase Locking (2PL)

- Two-phase locking protocol
 - All locking operations precede the first unlock operation in the transaction
 - Phases
 - Expanding (growing) phase
 - New locks can be acquired but none can be released
 - Lock conversion upgrades (from read-locked to write-locked) must be done during this phase
 - Shrinking phase
 - Existing locks can be released but none can be acquired
 - Downgrades (from write-locked to read-locked) must be done during this phase

Guaranteeing Serializability by Two-Phase Locking (2PL)

- If every transaction in a schedule follows the two-phase locking protocol, schedule guaranteed to be serializable
- Two-phase locking may limit the amount of concurrency that can occur in a schedule
- Some serializable schedules will be prohibited by two-phase locking protocol

Variations of Two-Phase Locking (2PL)

- Basic 2PL
 - Technique described on previous slides
- Conservative (static) 2PL
 - Requires a transaction to lock all the items it accesses before the transaction begins
 - Predeclare read-set and write-set
 - Deadlock-free protocol
 - difficult to use in practice because of the need to predeclare the read-set and write-set, which is not possible in some situations.
- Strict 2PL
 - Transaction does not release exclusive locks (write locks) until after it commits or aborts. It is not deadlock-free.

Variations of Two-Phase Locking (cont'd.)

- Rigorous 2PL
 - Transaction does not release any locks (exclusive or shared) until after it commits or aborts
- Concurrency control subsystem responsible for generating read_lock and write_lock requests
- Locking generally considered to have high overhead
 - because every read or write operation is preceded by a system locking request

(a)

T_1	T_2
read_lock(Y); read_item(Y); unlock(Y); write_lock(X); read_item(X); $X := X + Y$; write_item(X); unlock(X);	read_lock(X); read_item(X); unlock(X); write_lock(Y); read_item(Y); $Y := X + Y$; write_item(Y); unlock(Y);

Figure 21.4

Transactions T_1' and T_2' , which are the same as T_1 and T_2 in Figure 21.3 but follow the two-phase locking protocol. Note that they can produce a deadlock.

Figure 21.3

Transactions that do not obey two-phase locking.
(a) Two transactions T_1 and T_2 .

T_1'	T_2'
read_lock(Y); read_item(Y); write_lock(X); unlock(Y) read_item(X); $X := X + Y$; write_item(X); unlock(X);	read_lock(X); read_item(X); write_lock(Y); unlock(X) read_item(Y); $Y := X + Y$; write_item(Y); unlock(Y);

Dealing with Deadlock and Starvation

- Deadlock
 - Occurs when each transaction T in a set is waiting for some item locked by some other transaction T'
 - Both transactions stuck in a waiting queue

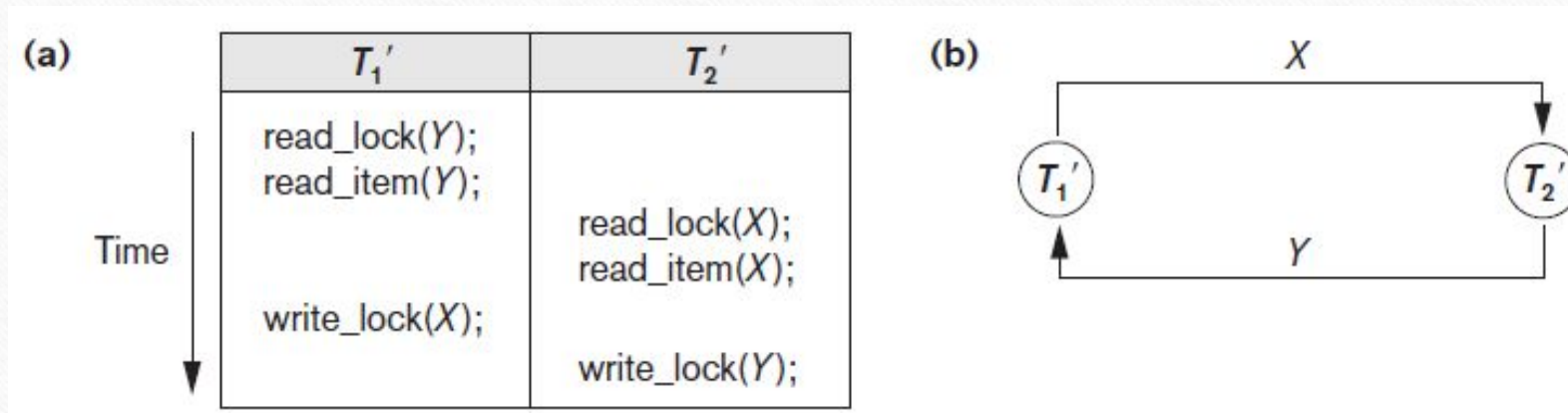


Figure 21.5 Illustrating the deadlock problem (a) A partial schedule of T_1' and T_2' that is in a state of deadlock (b) A wait-for graph (WFG) for the partial schedule in (a)

Dealing with Deadlock and Starvation (cont'd.)

- Deadlock prevention protocols
 - Every transaction locks all items it needs in advance
 - Ordering all items in the database
 - Transaction that needs several items will lock them in that order
 - Both approaches impractical

Dealing with Deadlock and Starvation (cont'd.)

- A number of other deadlock prevention schemes have been proposed that make a decision about what to do with a transaction involved in a possible deadlock situation:
- these techniques use the concept of **transaction timestamp** $TS(T)$, which is a unique identifier assigned to each transaction.
- if transaction T_1 starts before transaction T_2 ,
then $TS(T_1) < TS(T_2)$.

NOTE: *older* transaction (which starts first) has the smaller timestamp value

Dealing with Deadlock and Starvation (cont'd.)

- 2 Protocols based on a timestamp
- Suppose that transaction T_i tries to lock an item X but is not able to because X is locked by some other transaction T_j
 - **Wait-die.** If $TS(T_i) < TS(T_j)$, then (T_i older than T_j) T_i is allowed to wait; otherwise (T_i younger than T_j) abort T_i (T_i dies) and restart it later *with the same timestamp*.
- - **Wound-wait.** If $TS(T_i) < TS(T_j)$, then (T_i older than T_j) abort T_j (T_i wounds T_j) and restart it later *with the same timestamp*; otherwise (T_i younger than T_j) T_i is allowed to wait.
- In wound-wait, transactions only wait for older transactions so no cycle is created.

Dealing with Deadlock and Starvation (cont'd.)

- No waiting algorithm
 - If transaction unable to obtain a lock, immediately aborted and restarted later
- Cautious waiting algorithm
 - **Cautious waiting.** If T_j is not blocked (not waiting for some other locked item), then T_i is blocked and allowed to wait; otherwise abort T_i .
- Deadlock detection
 - System checks to see if a state of deadlock exists
 - Wait-for graph (a state of deadlock if and only if the wait-for graph has a cycle)

Dealing with Deadlock and Starvation (cont'd.)

- Victim selection
 - Deciding which transaction to abort in case of deadlock (avoid selecting transactions that have been running for a long time)
- Timeouts
 - If system waits longer than a predefined time, it aborts the transaction
- Starvation
 - Occurs if a transaction cannot proceed for an indefinite period of time while other transactions continue normally
 - Solution: first-come-first-served queue

21.2 Concurrency Control Based on Timestamp Ordering

- Timestamp
 - Unique identifier assigned by the DBMS to identify a transaction
 - Assigned in the order submitted
 - Transaction start time
- Concurrency control techniques based on timestamps do not use locks, so
 - Deadlocks cannot occur

Concurrency Control Based on Timestamp Ordering (cont'd.)

- Generating timestamps
 - Counter incremented each time its value is assigned to a transaction
 - Current date/time value of the system clock
 - Ensure no two timestamps are generated during the same tick of the clock
- General approach
 - Enforce equivalent serial order on the transactions based on their timestamps

Concurrency Control Based on Timestamp Ordering (cont'd.)

- Timestamp ordering (TO)
 - Allows interleaving of transaction operations
 - Must ensure timestamp order is followed for each pair of conflicting operations
- Each database item assigned two timestamp values
 - 1) **read_TS(X)**
 1. **read_TS(X).** The **read timestamp** of item X is the largest timestamp among all the timestamps of transactions that have successfully read item X —that is, $\text{read_TS}(X) = \text{TS}(T)$, where T is the *youngest* transaction that has read X successfully.
 - 2) **write_TS(X)**
 2. **write_TS(X).** The **write timestamp** of item X is the largest of all the timestamps of transactions that have successfully written item X —that is, $\text{write_TS}(X) = \text{TS}(T)$, where T is the *youngest* transaction that has written X successfully. Based on the algorithm, T will also be the last transaction to write item X , as we shall see.

Concurrency Control Based on Timestamp Ordering (cont'd.)

- Basic TO algorithm

1. Whenever a transaction T issues a `write_item(X)` operation, the following check is performed:
 - a. If $\text{read_TS}(X) > \text{TS}(T)$ or if $\text{write_TS}(X) > \text{TS}(T)$, then abort and roll back T and reject the operation. This should be done because some *younger* transaction with a timestamp greater than $\text{TS}(T)$ —and hence *after* T in the timestamp ordering—has already read or written the value of item X before T had a chance to write X , thus violating the timestamp ordering.
 - b. If the condition in part (a) does not occur, then execute the `write_item(X)` operation of T and set $\text{write_TS}(X)$ to $\text{TS}(T)$.
2. Whenever a transaction T issues a `read_item(X)` operation, the following check is performed:
 - a. If $\text{write_TS}(X) > \text{TS}(T)$, then abort and roll back T and reject the operation. This should be done because some younger transaction with timestamp greater than $\text{TS}(T)$ —and hence *after* T in the timestamp ordering—has already written the value of item X before T had a chance to read X .
 - b. If $\text{write_TS}(X) \leq \text{TS}(T)$, then execute the `read_item(X)` operation of T and set $\text{read_TS}(X)$ to the *larger* of $\text{TS}(T)$ and the current $\text{read_TS}(X)$.

Concurrency Control Based on Timestamp Ordering (cont'd.)

- Thomas's write rule
 - Modification of basic TO algorithm
 - Does not enforce conflict serializability
 - Rejects fewer write operations by modifying checks for write_item(X) operation